



TesseraApp

IMPLEMENTATION 2 · PHASE 2 (FINAL)

Software System Implementation & UI Design

The complete, hardened system explained module by module and seam by seam — how the code is organised, how each security-critical path works in code, and what changed since the Phase 1 prototype.

Phase 2 — final

Code walkthrough

Backend + frontend

Companion: Implementation video

AUTHORS

Robert C. Oliver, Jr. · Travis L. Lester

COURSE · ADVISOR

CPSC 69100 Master's Project · Prof. Larry
Motuzis

INSTITUTION

Lewis University

DATE · VERSION

2026 · v1.0

What this report covers

This is the **Implementation 2 (Phase 2, final)** report for TesseractApp — a hybrid, zero-trust Customer Identity & Access Management (CIAM) platform. Where Phase 1 proved the concept end-to-end, Phase 2 documents the *realised* system: the actual modules, the security seams, the data layer, the frontend, and how they fit together.

It is written to be read alongside the code and the accompanying implementation video. Every section names the concrete classes involved and, where the logic is load-bearing, shows the passage of code that carries the weight and explains why it is written the way it is.

Contents

1	Introduction & the complete system	03
2	How the code is organised	04
3	Two deliberate persistence strategies	05
4	Security implementation — the filter chain	06
5	Token provider — minting & verifying JWTs	07
6	Refresh rotation & reuse detection	08
7	Authenticator-app MFA (TOTP)	09
8	Federated login (OAuth2 / OIDC)	10
9	Data layer & schema management	11
10	Frontend implementation (Angular 21)	12
11	Build, packaging & deployment	13
12	Testing & quality	14
13	Changes since Phase 1 · limitations	15
14	User interface & implementation video	16

01 Introduction & the complete system

Phase 2 completes the security feature set, consolidates the data layer, and hardens the build — turning the Phase 1 prototype into a coherent, documented, deployable system.

1.1 Phase 2 objectives

ID	OBJECTIVE	STATUS
P2-1	Complete the security set — refresh rotation, reuse detection, TOTP MFA, org-scoped admin	Done
P2-2	Consolidate schema management into an idempotent script	Done
P2-3	Comprehensive developer & API documentation	Done
P2-4	Build / packaging via multi-stage Docker	Done
P2-5	Federated login (Google / GitHub / Microsoft) — code complete	Done
P2-6	Automated test suite for the security seams	Partial
P2-7	Real SMS delivery · AWS CI/CD · externalised config	Future

1.2 The complete system

The final system implements the full SRS scope behind one auditable, revocable identity core:

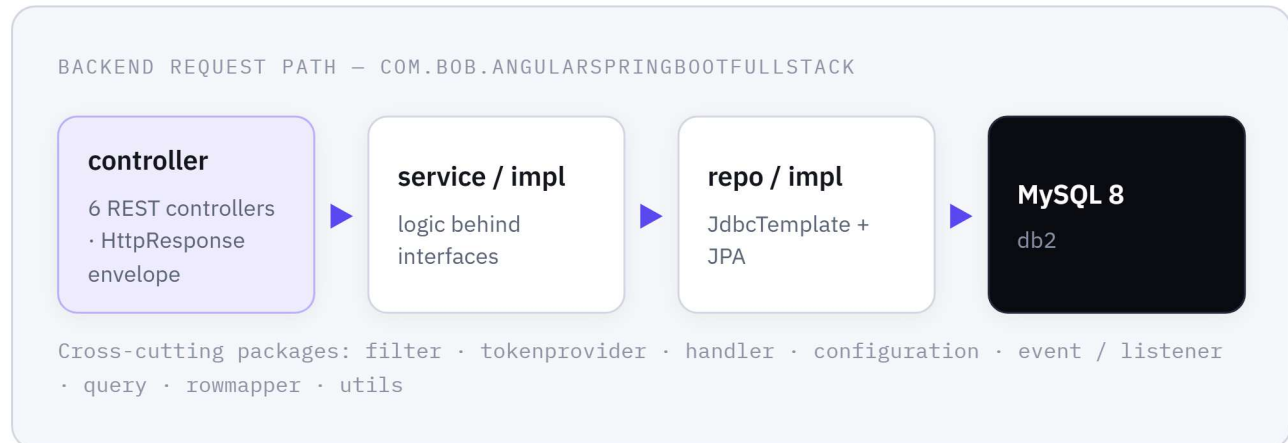
- **Authentication:** registration + email verification, login, logout, password reset, stateless JWT access tokens, server-tracked rotating refresh sessions with reuse detection, and per-account brute-force throttling.
- **MFA:** authenticator-app TOTP (QR enrollment, challenge-bound verification, single-use SHA-256 recovery codes); SMS 2FA (delivery stubbed — code logged, not sent, to avoid Twilio charges in development).
- **Federation:** OAuth2 / OIDC login (Google, GitHub, Microsoft) with find-or-create account linking — code complete, awaiting real provider credentials to go live.
- **Authorization:** seven-role, permission-based RBAC enforced at URL *and* method level, with organization-scoped administration.
- **Administration & audit:** user directory, role reassignment, account-state control — all written to an immutable audit trail of 15 event types via an event-driven pipeline.
- **Business domain:** customers & invoices CRUD, dashboard statistics, XLSX export.
- **Self-service:** profile, password, MFA, and session / device management.

THE ONE-SENTENCE ARCHITECTURE

Every authentication path — in-house password, TOTP-verified, or federated OAuth2 — converges on a **single token-issuance seam** (SessionServiceImpl), so RBAC, MFA policy, and audit logging apply identically no matter how a user signs in.

02 How the code is organised

A strictly layered Spring Boot backend and a standalone-component Angular frontend, deployed as a single container.



PACKAGE	RESPONSIBILITY
<code>controller/</code>	6 REST controllers; every response wrapped in the <code>HttpResponse</code> envelope (timestamp, status, code, message, data).
<code>service/</code> · <code>serviceimpl/</code>	Business logic behind interfaces — <code>UserService</code> , <code>SessionService</code> , <code>TotpService</code> , <code>EventService</code> , <code>NotificationService</code> , <code>FederatedIdentityService</code> .
<code>repo/</code> · <code>repoimpl/</code>	<code>NamedParameterJdbcTemplate</code> for identity; Spring Data JPA for the business domain.
<code>query/</code> · <code>rowmapper/</code>	SQL string constants and <code>ResultSet</code> → POJO mappers for the JDBC layer.
<code>filter</code> · <code>tokenprovider</code>	<code>CustomAuthFilter</code> and <code>TokenProvider</code> — the per-request security seam.
<code>configuration</code> · <code>handler</code>	<code>SecurityConfig</code> , OAuth2 client config; custom 401 / 403 handlers.
<code>event/</code> · <code>listener/</code>	Event-driven audit pipeline (<code>UserEvent</code> published, listener persists off the hot path).

Frontend (`securecapitaapp/src/app/`): `app.config.ts` (providers), `app.routes.ts` (lazy routes + guards), and folders `features/` (login, home, customers, invoices, security-center, users, profile), `service/`, `guard/`, `interceptor/`, `enum/` and `shared/`. No `NgModule` — every component is standalone.

03 Two deliberate persistence strategies

The codebase uses **two** persistence approaches on purpose, split by risk profile — not by accident of history.

DOMAIN	STRATEGY	WHY
Identity & auth users, roles, sessions, TOTP, events	Hand-written, named-parameter SQL via NamedParameterJdbcTemplate	Every query that touches identity is explicit and auditable — no ORM lazy-loading or cascade surprises in the security path.
Business domain customers, invoices, services	Spring Data JPA / Hibernate	Lower-stakes, CRUD-heavy work where the ORM's convenience and derived queries win.

Notably, the User model is a plain POJO mapped by a hand-written RowMapper — it is *not* a JPA entity. The SQL lives as constants in the query/ package; the mapper turns a ResultSet row into the domain object:

```
// rowmapper/UserRowMapper.java — explicit, no reflection magic
public User mapRow(ResultSet rs, int rowNumber) throws SQLException {
    return User.builder()
        .id(rs.getLong("id"))
        .email(rs.getString("email"))
        .password(rs.getString("password"))           // BCrypt hash — never leaves the
service layer
        .enabled(rs.getBoolean("enabled"))
        .isNotLocked(rs.getBoolean("non_locked"))
        .usingMfa(rs.getBoolean("using_mfa"))
        .passwordChangedAt(rs.getTimestamp("password_changed_at"))
        .build();
}
```

```
// repoimpl/UserRepositoryImpl.java — named parameters, never string concatenation
User user = jdbc.queryForObject(SELECT_USER_BY_EMAIL_QUERY,
    Map.of("email", email), new UserRowMapper());
```

Named parameters make SQL injection structurally impossible in this layer — user input is always bound, never interpolated — and keep the audited security queries readable in one place.

04 The filter chain

SecurityConfig defines a single SecurityFilterChain: CSRF disabled (there is no server session to forge against), CORS whitelisted, sessions STATELESS, authority rules ordered most-specific first, OAuth2 login wired to the federated success handler, and custom 401 / 403 handlers. The custom filter runs *before* Spring's username/password filter.

```
// configuration/SecurityConfig.java
@Bean SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http.csrf(AbstractHttpConfigurer::disable)           // stateless API – no CSRF token
    .cors(withDefaults())
    .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
    .authorizeHttpRequests(auth -> auth
        .requestMatchers(PUBLIC_ROUTES).permitAll()
        .requestMatchers(DELETE, "/user/delete/**").hasAuthority("DELETE:USER")
        .requestMatchers(DELETE, "/customer/delete/**").hasAuthority("DELETE:CUSTOMER")
        .anyRequest().authenticated())
    .oauth2Login(o -> o.successHandler(oAuth2LoginSuccessHandler))
    .exceptionHandling(e -> e
        .authenticationEntryPoint(customAuthenticationEntryPoint) // 401
        .accessDeniedHandler(customAccessDeniedHandler));         // 403
    http.addFilterBefore(customAuthFilter, UsernamePasswordAuthenticationFilter.class);
    return http.build();
}
```

The per-request filter skips public routes, validates the bearer token, and sets the security context **only when the token carries authorities** — so a refresh token, which carries none, can never authenticate an API call.

```
// filter/CustomAuthFilter.java
protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain
chain) {
    if (isPublic(req)) { chain.doFilter(req, res); return; }
    String token = extractToken(req);           // strips the "Bearer " prefix
    String email = tokenProvider.getSubject(token, req);
    if (tokenProvider.isTokenValid(email, token)) {
        List<GrantedAuthority> authorities = tokenProvider.getAuthorities(token);
        if (authorities != null) {               // null ⇒ a refresh token ⇒ not
authenticated
            Authentication auth = tokenProvider.getAuthentication(email, authorities, req);
            SecurityContextHolder.getContext().setAuthentication(auth);
        }
    }
    chain.doFilter(req, res);
}
```

ROUTE GUARDS ARE NOT THE BOUNDARY

The Angular authenticationGuard / adminGuard are usability aids only. Authorization is enforced **server-side** on every request by this filter plus @PreAuthorize checks — a bypassed frontend guard changes nothing.

05 Minting & verifying JWTs

TokenProvider mints and verifies HMAC-SHA-512 JWTs. **Access** tokens (30 min) carry the user's authorities and a session id (sid); **refresh** tokens (5 day) carry only a token id (jti) and sid — deliberately no authorities, so they cannot be replayed as access tokens.

```
// tokenprovider/TokenProvider.java
public String createAccessToken(UserPrincipal user) {
    return JWT.create()
        .withIssuer(TESSERA).withAudience(CUSTOMER_MANAGEMENT_SERVICE)
        .withIssuedAt(new Date()).withSubject(user.getUsername())
        .withArrayClaim(AUTHORITIES, authoritiesOf(user)) // access token ONLY
        .withClaim(SESSION_ID, user.getSessionId())
        .withExpiresAt(Date.from(now().plus(30, MINUTES)))
        .sign(HMAC512(secret.getBytes()));
}
```

Validity is more than a signature and an expiry check. The passwordChangedAt guard makes a password change silently invalidate every token issued before it — a token whose issuedAt predates the user's last password change is rejected even though its signature is perfectly valid.

```
// a password change must kill outstanding tokens, everywhere, at once
public boolean isTokenValid(String email, String token) {
    JWTVerifier verifier = verifier();
    if (!verifier.verify(token)...) return false; // signature + expiry
    Date issuedAt = decoded.getIssuedAt();
    Date changedAt = userService.getPasswordChangedAt(email);
    return changedAt == null || issuedAt.after(changedAt); // stale-before-change ⇒ invalid
}
```

TOKEN	LIFETIME	CLAIMS	PURPOSE
access	30 min	sub, authorities, sid	Sent on every API request; validated by signature alone (no DB hit).
refresh	5 day	sub, jti, sid	Exchanged for a new access token; every use is a tracked DB row.

06 Refresh rotation & reuse detection

SessionServiceImpl is the single token-issuance seam. `issueTokenPair()` opens a session *family*; `rotate()` exchanges a refresh token for a fresh pair, rotating the jti and detecting replay. A refresh presents its jti; the row is looked up. If it is already superseded or revoked, that is a replay — the whole family is revoked, a `TOKEN_REUSE_DETECTED` event is audited, and the request is rejected.

```
// serviceimpl/SessionServiceImpl.java — NOT @Transactional (see callout)
RefreshSession session = sessionRepo.findByJti(jti)
    .orElseThrow(() -> new ApiException("Session not found"));
if (session.isSuperseded() || session.isRevoked()) {
    handleReuse(session); // revoke whole family + publish audit event
    throw new ApiException("This session has been revoked. Please sign in again.");
}
jdbc.update(SUPERSEDE_SESSION_QUERY, Map.of("id", session.getId()));
String newJti = UUID.randomUUID().toString();
insertSessionRow(session.getUserId(), session.getFamily(), newJti, request); // then insert
```

SPA	/user/refresh/token	SessionService	MySQL
1	Access token expires; the interceptor catches the 401 and posts the stored refresh token. SPA → /user/refresh/token		
2	The handler looks up the row by jti. SessionService → MySQL		
3	Fresh token: supersede the old row, insert a new jti in the same family, return a new access + refresh pair. SessionService → MySQL → SPA		
4	Replayed token: the row is already superseded → revoke the entire family and audit <code>TOKEN_REUSE_DETECTED</code> . SessionService → MySQL		
5	Reject with a generic message; the user (and the thief) must sign in again. SessionService → SPA		

LOAD-BEARING DECISION — COMMIT-THEN-THROW

`rotate()` is deliberately **not** `@Transactional`: the family revocation must *commit* before the rejection throws. Inside a transaction the throw would roll the revocation back — un-punishing the exact replay it just caught. Supersede-then-insert ordering means a crash mid-way leaves an already-retired token, never two live tokens in one family.

07 Authenticator-app MFA (TOTP)

TotpServiceImpl implements RFC 6238 in-house (TotpUtils); only QR rasterization uses the ZXing library. Enrollment generates a Base32 secret, shows a QR code, and requires one correct code to confirm before MFA is switched on. Login verification is **challenge-bound**: a first-factor success mints a short-lived opaque challenge (5 min), and the verify endpoint refuses any code that is not tied to a live challenge — so possessing the authenticator alone can never skip the password.

```
// serviceimpl/TotpServiceImpl.java – verify is bound to a live challenge
MfaChallenge challenge = challengeRepo.findActive(challengeId)
    .orElseThrow(() -> new ApiException("Challenge expired. Please sign in again.));
if (!TotpUtils.verifyCode(secretFor(challenge.getUserId()), submittedCode)) {
    throw new ApiException("Invalid code"); // wrong code does NOT consume the challenge
}
challengeRepo.consume(challengeId); // correct code burns the challenge exactly once
return sessionService.issueTokenPair(challenge.getUserId(), request); // converge on the one
seam
```

Recovery codes are shown once at enrollment, stored only as **SHA-256 hashes**, and burned atomically — the affected-row count of the UPDATE is the verdict, so a code cannot be used twice even under concurrent requests.

```
// single-use recovery code – the DB does the concurrency control
int burned = jdbc.update(BURN_RECOVERY_CODE_QUERY,
    Map.of("userId", userId, "hash", sha256(submitted)));
if (burned == 0) throw new ApiException("Invalid or already-used recovery code");
```

SMS 2FA shares the same challenge machinery; only the delivery call is stubbed (the code is logged, not sent) to avoid Twilio charges in development. Un-stubbing is a one-line change gated behind an SMS_ENABLED flag — see the companion `final_implementation_guide.md` §1.

08 Federated login (OAuth2 / OIDC)

Spring Security's OAuth2 Client runs the Authorization Code flow with each provider. When it completes, OAuth2LoginSuccessHandler is the token-exchange point: it normalizes per-provider userinfo, does a find-or-create on (provider, subject), applies the same account-state and MFA policy as in-house login, then issues *our own* JWT via SessionService.

SPA	Provider	SuccessHandler	MySQL
1	User clicks “Continue with Google/GitHub/Microsoft”; Spring redirects to the provider. SPA → Provider		
2	Provider authenticates, calls back with an authorization code; Spring exchanges it for userinfo. Provider → SuccessHandler		
3	Handler extracts the stable subject (Google/Microsoft sub; GitHub numeric id) and does find-or-create. SuccessHandler → MySQL		
4	Applies account-state + MFA parity, then issues our JWT and redirects the SPA with tokens in the URL fragment . SuccessHandler → SPA		

```
// handler/OAuth2LoginSuccessHandler.java – per-provider normalisation, then converge
String provider = token.getAuthorizedClientRegistrationId();
String subject = "github".equals(provider) ? attr("id") : attr("sub");
User user = federatedIdentityService.findOrCreate(provider, subject, profile);
if (!user.isEnabled() || !user.isNotLocked()) redirect("/login?error=account");
else if (user.isUsingMfa()) redirect("/oauth2/callback#mfa=totp&challenge=" + challenge);
else {
    // same seam as password + TOTP login
    TokenPair pair = sessionService.issueTokenPair(user.getId(), request);
    redirect(uiUrl + "/oauth2/callback#access_token=" + pair.access() + "&refresh_token=" +
pair.refresh());
}
```

Tokens ride in the URL **fragment** because fragments are never sent to servers and stay out of access logs. Every failure degrades to a coarse ?error= redirect — never disclosing whether an account exists (OWASP OAT-007). Only (provider, subject) and a minimal profile are persisted; no third-party password or token is ever stored.

09 Data layer & schema management

- **Identity / auth tables** (users, roles, user–role, organizations + membership, OAuth provider links, verifications, events + user-events, TOTP credentials + recovery codes, MFA challenges, refresh sessions) are defined in an idempotent schema.sql and accessed through JdbcTemplate.
- **Business tables** (customer, invoice, services, invoiceserviceitems) are JPA-managed by Hibernate (ddl-auto: update in development).
- A prior **Flyway** migration set was **removed** after its baseline bookkeeping repeatedly blocked startup; the cumulative schema now lives in one idempotent script.

```
-- schema.sql – safe to run on every boot; never destroys data
CREATE TABLE IF NOT EXISTS RefreshSessions (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id     BIGINT NOT NULL,
  family      VARCHAR(36) NOT NULL,      -- one family per login
  jti         VARCHAR(36) NOT NULL UNIQUE, -- current token id in the family
  device      VARCHAR(255), ip VARCHAR(45),
  superseded  BOOLEAN DEFAULT FALSE,
  revoked     BOOLEAN DEFAULT FALSE,
  created_at  TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  expires_at  TIMESTAMP NOT NULL,
  CONSTRAINT fk_session_user FOREIGN KEY (user_id) REFERENCES Users (id)
);
INSERT INTO Roles (name, permission) VALUES ('ROLE_USER', 'READ:USER,READ:CUSTOMER')
ON DUPLICATE KEY UPDATE permission = VALUES(permission); -- idempotent seed
```

Referential integrity is enforced by foreign keys; single-use keys (verification, reset, recovery) are unique and expiring. A DemoDataSeeder defensively seeds the six demo accounts and sample business data on an empty database, so a fresh clone is immediately demonstrable.

PRODUCTION NOTE

For the cloud target, switch Hibernate to ddl-auto: validate and treat schema.sql as the single source of truth, so the deployed schema is reviewed and version-controlled rather than inferred at runtime.

10 Frontend implementation (Angular 21)

Standalone components bootstrapped from app.config.ts — no NgModule. The load-bearing client-side piece is the **token interceptor**, which attaches the access token and performs a **single-flight silent refresh** on a 401: the first 401 triggers one refresh; concurrent 401s queue on a shared BehaviorSubject and replay once the new token arrives, so a burst of requests never fires a storm of refreshes.

```
// interceptor/token.interceptor.ts — one refresh serves every concurrent 401
if (error.status === 401 && !isRefreshCall(req)) {
  if (!this.refreshing) {
    this.refreshing = true;
    this.refreshSubject.next(null);
    return this.userService.refreshToken$.pipe(switchMap(res => {
      this.refreshing = false;
      this.refreshSubject.next(res.data.access_token);
      return next(this.withToken(req, res.data.access_token));
    }));
  }
  return this.refreshSubject.pipe(
    // wait for the single in-flight refresh
    filter(t => t !== null), take(1),
    switchMap(t => next(this.withToken(req, t))));
}
```

PIECE	ROLE
authenticationGuard	Blocks routes without a valid JWT — a UX aid; the backend remains the boundary.
adminGuard	Hides admin routes from non-staff; server still enforces authority.
DataState<T>	An enum + wrapper driving explicit LOADING / LOADED / ERROR rendering on every async view.
ngx-toastr	Non-blocking toast feedback; consistent dismissible error alerts.

Theming is a single data-bs-theme attribute on <html>, persisted to localStorage so the dark/light choice survives reloads and sessions.

11 Build, packaging & deployment

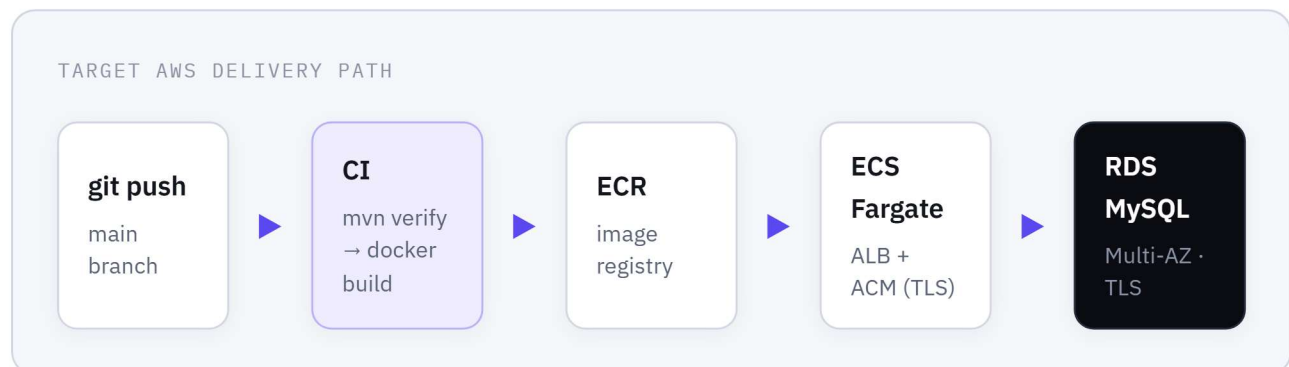
Maven builds the backend; the Angular CLI builds the frontend. A **three-stage Dockerfile** compiles the SPA, bakes it into the Spring Boot JAR's static resources, and runs it on a slim JRE with a health check — one self-contained, stateless image.

```
# 1 - build the Angular SPA
FROM node:20 AS web
COPY securecapitaapp/ .
RUN npm ci && npm run build -- --configuration production

# 2 - package the JAR with the SPA baked into static/
FROM maven:3.9-eclipse-temurin-21 AS api
COPY pom.xml . ; RUN mvn -q dependency:go-offline
COPY src/ src/
COPY --from=web /web/dist/ src/main/resources/static/
RUN mvn -q -Pprod clean package -DskipTests

# 3 - slim runtime, health-checked
FROM eclipse-temurin:21-jre
COPY --from=api /api/target/*.jar app.jar
HEALTHCHECK CMD curl -f http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java", "-jar", "/app.jar"]
```

A legacy Azure DevOps pipeline (build → push to ACR → App Service) exists. The Phase 2 *target* is a full AWS pipeline: image to ECR, service on ECS Fargate behind an ALB with ACM TLS, secrets in Secrets Manager, and managed MySQL on RDS. The migration plan — one-time setup, pipeline stages, and required prod environment — is detailed in the companion `final_implementation_guide.md` §3.



12 Testing & quality

- **Security tooling:** OWASP dependency-check in the Maven build (fails on CVSS ≥ 7); ESLint + Prettier on the frontend; a build-breaking JwtSecretGuard that refuses to start on a weak or missing JWT secret.
- **Automated tests (present):** a focused set over the security-critical paths — a login-enumeration test, the global exception handler, a customer-service test, and a JPA schema-sync test.
- **Manual verification:** every primary flow exercised end-to-end against a live database (evidenced by the demo videos) — register → verify → login → MFA → dashboard; admin role reassignment with audit; customer/invoice lifecycle with export; and the password-change-invalidates-all-tokens guarantee.

THE MOST SIGNIFICANT GAP

Automated coverage is **partial**. The top future-work item is a unit / integration suite for the security seams — token rotation + reuse detection, passwordChangedAt invalidation, RBAC allow/deny at URL and method level, org-scope, TOTP enroll/verify, and the anti-enumeration guarantee — run in CI so the pipeline fails on a red test.

SEAM	TEST TO ADD	GATE
Refresh rotation	Replay a superseded jti → family revoked + TOKEN_REUSE_DETECTED	Integration
Password change	Old access token rejected after passwordChangedAt moves	Unit
RBAC	Method- and URL-level allow/deny per role	Integration
Anti-enumeration	Identical response & timing for known vs unknown email	Integration
Federation	Find-or-create is idempotent on repeat (provider, subject)	Integration

13 Changes since Phase 1 · limitations

13.1 What Phase 2 added

- Completed refresh-session **rotation + reuse detection** and the user-facing Security Center (list / revoke devices).
- Added **authenticator-app (TOTP) MFA** with QR enrollment, challenge-bound verify, and single-use recovery codes.
- Completed **OAuth2 federated login** for Google / GitHub / Microsoft (code complete).
- Added **organization-scoped administration** alongside the seven-role RBAC.
- Removed the migration tool; reconciled the live schema via an idempotent schema.sql.
- Authored a complete documentation set (architecture, security, API, database, configuration, deployment, developer guide) and packaged the system via multi-stage Docker.
- Hardened startup robustness (defensive demo-data seeding; fail-fast JWT-secret guard).

13.2 Limitations & future work

ITEM	PLAN	GUIDE REF
SMS delivery	Un-stub Twilio behind SMS_ENABLED; keep dev free	§1
Federation go-live	Validate against real Google/GitHub/Microsoft credentials	§2
AWS CI/CD	ECR → ECS/App Runner → RDS, automated on push, over TLS	§3
Server-side org-scope + rate limiting	Gate analytics/billing server-side; add a general limiter (429 + Retry-After)	§4
Config hardening	Externalise frontend API origin; move profile images to object storage	§5
Automated tests	A meaningful suite for the security seams, run in CI	§6

Every remaining item traces to an SRS requirement ID; the exact files to touch, configuration, and acceptance criteria are in the companion `final_implementation_guide.md`.

14 UI & implementation video

The seams above are enforced server-side, but the user meets them through a dark-first “security console” UI: the admin directory below is organization-scoped and every state change on it is audited by the pipeline in §2.

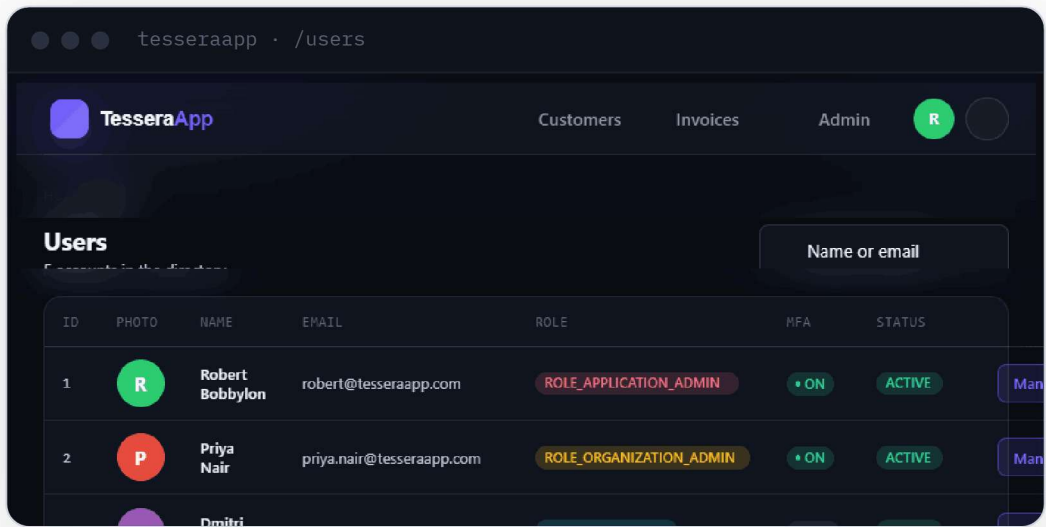


FIGURE 1 Organization-scoped admin user directory — role, MFA, and account-state, enforced server-side by the security seams in §4–§6.

Implementation video — segment plan

A video walks through the code live. Suggested segments (≈8 min total):

TIME	SEGMENT	SHOW
0:00	Tour the layout	Package tree (§2); the two persistence strategies (§3)
1:30	The filter chain	SecurityConfig + CustomAuthFilter (§4)
3:00	Rotation live	Log in, refresh, replay a token → family revoked (§6)
5:00	MFA	Enroll authenticator, challenge-bound verify (§7)
6:30	Federation	OAuth2LoginSuccessHandler converging on the seam (§8)
7:30	Ship it	Three-stage Docker build; health check (§11)

COMPANION DOCUMENTS

Read alongside the *Final Report / Combined Paper* (problem, literature, SRS, architecture) and the `final_implementation_guide.md` (the remaining-work development plan).

AI Tool Disclosure

AI tools, specifically Claude (Anthropic), were used to assist with drafting, structuring, and formatting this and other course deliverables throughout this project, as well as with code generation for select implementation areas, including security-related features. All architectural decisions, system design choices, implementation direction, and academic analysis reflect our own thinking and direction. We are disclosing this use proactively, in accordance with Lewis University's Academic Integrity policy.

AUTHORS

Robert C. Oliver, Jr. · Travis L. Lester

COURSE

CPSC 69100 — Master's Project

INSTITUTION

Lewis University